



计算力学期末作业报告

一维对流扩散方程的有限元求解与稳定化方法研究

学院： 建筑工程学院

专业： 流体力学

班级： 3 班

姓名： 陶杨

学号： 20252110004

日期： 2026 年 6 月 25 日

目录

1	引言.....	1
1.1	研究背景.....	1
1.2	研究内容.....	1
2	控制方程与基本理论.....	2
2.1	控制方程与边界条件.....	2
2.2	解析解.....	2
2.3	Peclet 数定义.....	2
3	有限元离散格式.....	3
3.1	标准 Galerkin 有限元.....	3
3.2	人工扩散与迎风格式.....	3
3.3	SUPG/Petrov-Galerkin 方法.....	3
4	程序实现.....	4
4.1	程序整体结构.....	4
4.2	关键实现说明.....	4
5	算例结果与分析.....	5
5.1	算例 1: $Pe=0.1$ (扩散主导).....	5
5.2	算例 2: $Pe=3.0$ (对流占优).....	6
6	总体刚度矩阵性质分析.....	7
6.1	对称性分析.....	7
6.2	正定性分析.....	7
6.3	振荡机理与稳定化本质.....	7
7	网格收敛性分析.....	8
7.1	误差统计.....	8
7.2	误差收敛规律.....	8
7.3	Galerkin 解的振荡演变.....	9
8	结论.....	10
9	附录.....	11
9.1	程序运行结果图.....	11
9.2	源程序代码.....	12

1 引言

1.1 研究背景

对流扩散方程是传热传质、计算流体力学、环境工程等领域的基础控制方程，描述了物理量在对流输运与扩散传递共同作用下的分布规律。当对流效应显著强于扩散效应（高 Peclet 数）时，标准 Galerkin 有限元方法会出现空间数值振荡，解失去物理意义，是有限元求解对流占优问题的核心难点。

目前主流的稳定化方法包括迎风差分、人工扩散法、SUPG 流线迎风 Petrov-Galerkin 法等，其中 SUPG 方法因兼顾稳定性与精度，成为应用最广泛的方案。本文基于两节点线性单元，实现三种离散格式并开展对比研究，深入分析对流占优问题的振荡机理与稳定化机制。

1.2 研究内容

本文完成以下研究工作：

1. 建立一维稳态对流扩散方程的线性单元有限元离散格式；
2. 实现标准 Galerkin、迎风格式、SUPG 三种稳定化方法；
3. 对比扩散主导、对流占优两种工况下各方法的数值表现与误差；
4. 分析总体刚度矩阵的对称性、正定性，阐明对流项的影响与振荡机理；
5. 开展网格收敛性研究，分析网格加密对数值振荡与计算精度的影响规律。

2 控制方程与基本理论

2.1 控制方程与边界条件

考虑区间 $[0, L]$ 上的一维稳态对流扩散控制方程：

$$-\kappa \frac{d^2 \theta}{dx^2} + v \frac{d\theta}{dx} = 0$$

其中： $\theta(x)$ 为待求标量场； $\kappa > 0$ 为扩散系数； $v > 0$ 为对流速度； 计算域长度 $L=1$ 。

边界条件为两端 Dirichlet 约束：

$$\theta(0) = 0, \quad \theta(L) = 1$$

2.2 解析解

上述方程的精确解析解为：

$$\theta_{exact}(x) = \frac{\exp\left(\frac{vx}{\kappa}\right) - 1}{\exp\left(\frac{vL}{\kappa}\right) - 1}$$

当全局 Peclet 数较大时，指数函数易出现数值溢出，程序中采用渐近形式进行稳定化处理，避免计算失效。

2.3 Peclet 数定义

单元 Peclet 数是衡量对流与扩散相对强度的无量纲数，定义为：

$$Pe = \frac{vl_e}{2\kappa}$$

其中 l_e 为单个单元长度。区分单元尺度对流/扩散强弱的临界值为 $Pe=1$ ：当 $Pe < 1$ 时，扩散效应占主导，数值求解稳定性好；当 $Pe > 1$ 时，对流效应占主导，标准有限元易出现空间振荡。

3 有限元离散格式

3.1 标准 Galerkin 有限元

对于两节点线性单元，标准 Galerkin 离散后的单元刚度矩阵由扩散项和对流项两部分叠加而成：

$$\mathbf{K}_e = \mathbf{K}_{diff} + \mathbf{K}_{conv}$$

扩散项对应二阶导数项，其单元刚度矩阵为对称正定矩阵：

$$\mathbf{K}_{diff} = \frac{\kappa}{l_e} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

对流项对应一阶导数项，其单元刚度矩阵为非对称矩阵：

$$\mathbf{K}_{conv} = \frac{v}{2} \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$$

3.2 人工扩散与迎风格式

针对对流占优问题的数值振荡，人工扩散法通过增大等效扩散系数恢复方程的椭圆性，等效扩散系数定义为：

$$\bar{\kappa} = \kappa + \alpha \cdot \frac{vl_e}{2}$$

其中 α 为稳定参数，用于控制人工扩散的强度。当 $\alpha=1$ 时对应迎风格式（单边差分），此时人工扩散强度最大，可完全消除数值振荡，但会引入过量的数值耗散，导致出口边界层被过度抹平，解的精度显著下降。

3.3 SUPG/Petrov-Galerkin 方法

流线迎风 Petrov-Galerkin（SUPG）方法选取最优稳定参数，在消除振荡的同时尽可能减小数值耗散，最优参数表达式为：

$$\alpha_{opt} = \coth(Pe) - \frac{1}{Pe}$$

最优稳定参数可随单元 Peclet 数实现自适应调整，当 Pe 趋近于 0 即扩散效应占主导时， α_{opt} 趋近于 0，SUPG 方法退化为标准 Galerkin 有限元格式；当 Pe 趋近于无穷大即处于强对流工况时， α_{opt} 趋近于 1，方法退化为迎风格式；值得注意的是，对于一维稳态对流扩散问题的两节点线性单元，采用该最优参数的 SUPG 方法具备节点精确性，节点处的数值解与解析解完全一致，计算误差仅来源于计算机浮点运算的舍入误差。

4 程序实现

4.1 程序整体结构

基于 Python+NumPy 开发有限元求解程序，采用模块化设计，核心功能模块如下：

1. 参数计算模块：计算 SUPG 最优稳定参数，处理低 Pe 时的数值奇异问题；
2. 单元分析模块：根据等效扩散系数生成单元刚度矩阵；
3. 矩阵组装模块：采用直接叠加法组装总体刚度矩阵；
4. 边界处理模块：采用行修改法施加 Dirichlet 边界条件，保留列耦合关系；
5. 求解模块：求解线性方程组，同步计算节点处解析解；
6. 后处理模块：绘制对比曲线、计算最大节点误差、分析矩阵性质；
7. 收敛分析模块：多网格密度下的误差统计与收敛曲线绘制。

4.2 关键实现说明

在程序实现层面，针对边界处理、数值稳定与精度评估均做了针对性设计：边界条件施加采用行修改策略，仅修改边界自由度对应的方程行而不清零边界列，保证内部节点与边界的耦合关系准确；数值稳定性方面，对大 Peclet 数工况下的解析解采用渐近表达形式，避免指数函数运算出现数值溢出；误差统计以解析解为基准，计算所有节点的最大绝对误差，以此量化不同离散方法的计算精度。

5 算例结果与分析

固定计算域 $L=1$ 、对流速度 $v=1$ 、单元数 $nel=20$ ，分别针对单元 $eclet$ 数 $Pe=0.1$ （扩散主导）和 $Pe=3.0$ （对流占优）两种工况开展计算，对比三种方法的数值表现。

5.1 算例 1： $Pe=0.1$ （扩散主导）

当单元 $Peclet$ 数为 0.1 时，扩散效应占绝对主导，三种方法的数值解均保持平滑。

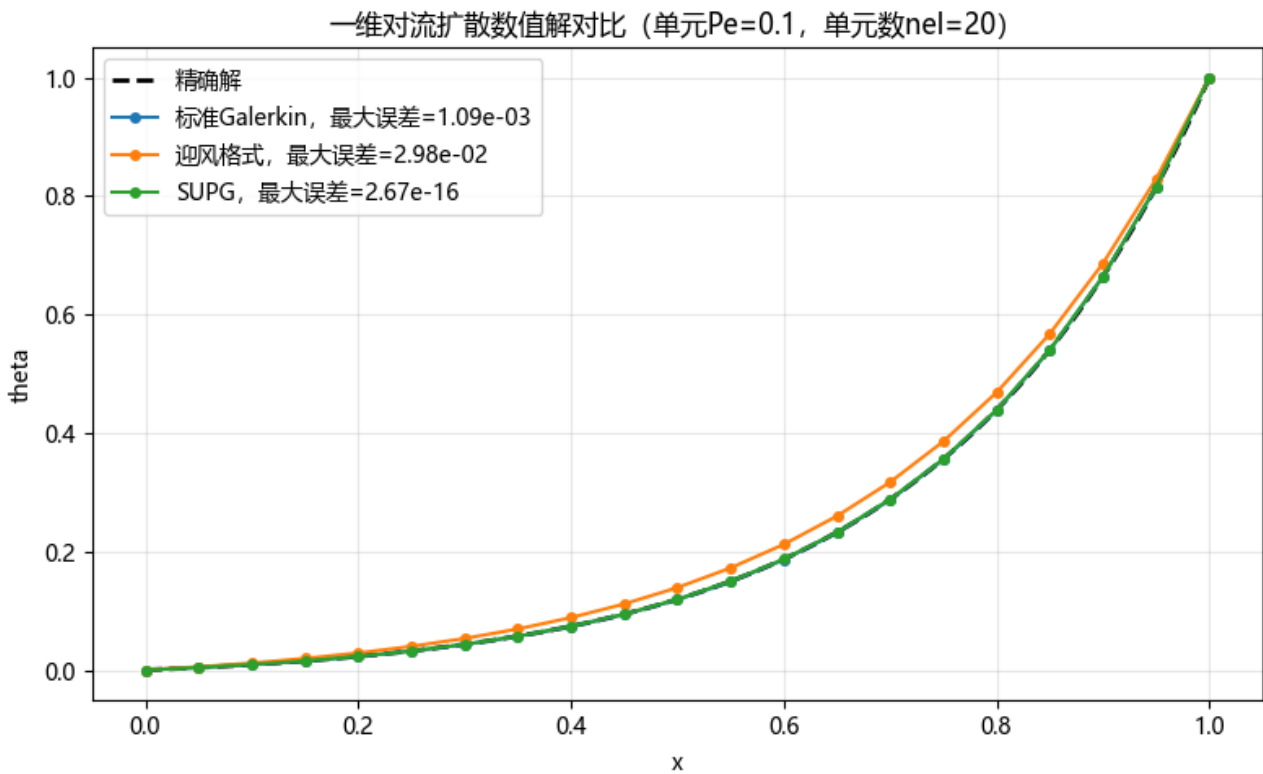


图 1 $Pe=0.1$ 时三种方法数值解与精确解对比

三种方法的最大节点误差统计如表格 1 所示。

表格 1 $Pe=0.1$ 工况下各方法最大节点误差

方法	最大节点误差
标准 Galerkin	1.094240e-03
迎风格式	2.977253e-02
SUPG	2.666553e-16

结果分析：

- 1. 扩散主导工况下，标准 Galerkin 方法无数值振荡，计算精度较高；
- 2. 迎风格式因引入额外人工扩散，误差略大于标准 Galerkin 方法；
- 3. SUPG 方法误差接近机器浮点精度，实现了节点精确解。

5.2 算例 2: $Pe=3.0$ (对流占优)

当单元 Peclet 数为 3.0 时, 对流效应占主导, 三种方法表现出显著的性能差异。

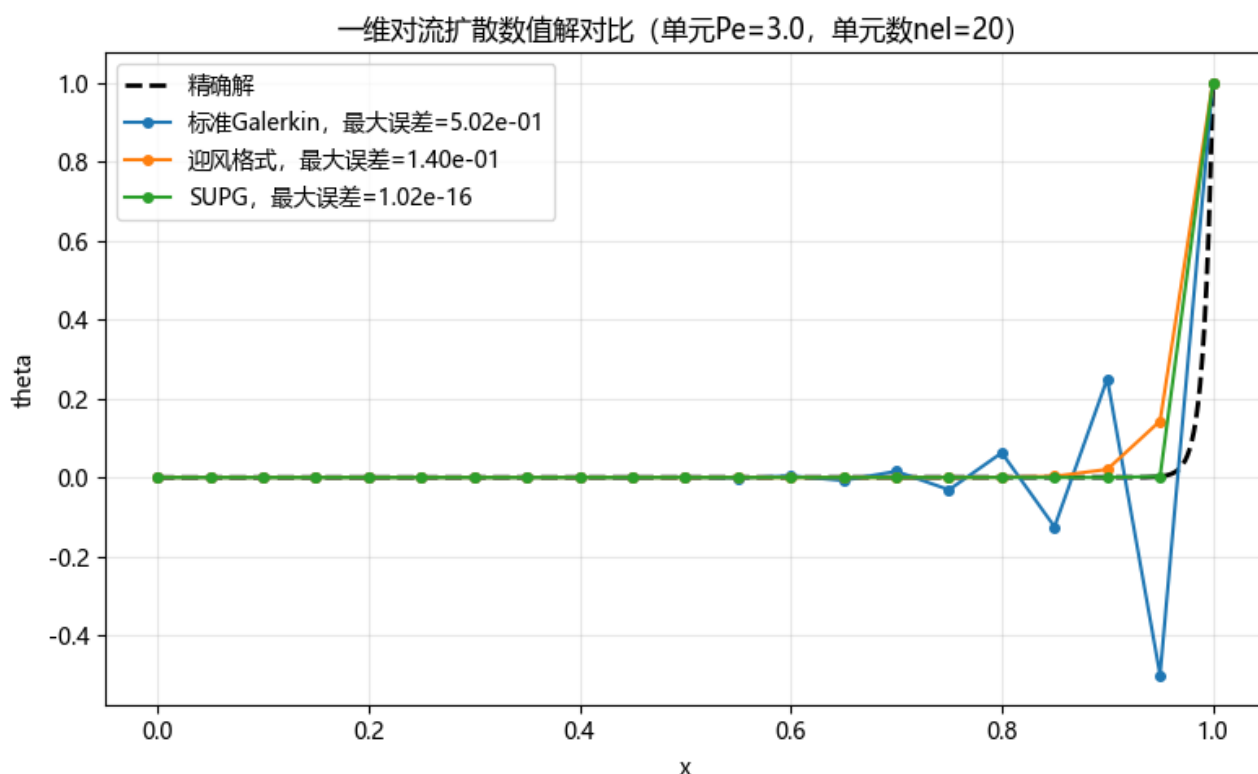


图 2 $Pe=3.0$ 时三种方法数值解与精确解对比

三种方法的最大节点误差统计如表格 2 所示。

表格 2 $Pe=3.0$ 时三种方法数值解与精确解对比

方法	最大节点误差
标准 Galerkin	5.024802e-01
迎风格式	1.403784e-01
SUPG	1.023487e-16

结果分析:

1. 标准 Galerkin 方法: 解出现明显的空间振荡, 误差最大, 失去物理意义。其本质是对流占优时双线性形式丧失椭圆性, 弱形式不适定, 导致数值解失稳。
2. 迎风格式: 通过引入人工扩散恢复了方程椭圆性, 完全消除了数值振荡, 但过量的数值耗散使得出口边界层被明显抹平, 解的精度下降明显。
3. SUPG 方法: 通过最优稳定参数自适应调整人工扩散强度, 既完全消除了数值振荡, 又最大程度保留了解的精度, 节点值与精确解几乎完全一致, 兼顾了稳定性与精度。

6 总体刚度矩阵性质分析

针对 $Pe=3.0$ 工况下未施加边界条件的标准 Galerkin 总体刚度矩阵，分析其对称性、正定性，阐明对流项对矩阵性质的影响与数值振荡的数学根源。

6.1 对称性分析

扩散项刚度矩阵为对称矩阵，对流项刚度矩阵为非对称矩阵，二者叠加后的总体刚度矩阵为非对称矩阵。数值计算结果显示，矩阵最大不对称差值为 1.0×10^0 ，验证了对流项是矩阵非对称的核心原因。

6.2 正定性分析

未施加边界条件时，总体刚度矩阵存在零特征值（对应常数齐次解），矩阵呈半正定状态。对流项的存在会持续削弱矩阵的正定性，当对流占优、Peclet 数足够大时，矩阵会出现负实特征值，双线性形式丧失椭圆性，这是数值振荡的数学本质。

6.3 振荡机理与稳定化本质

从弱形式角度，标准 Galerkin 方法的权函数与试探函数对称，未考虑对流问题信息单向传播的特性，上下游权重一致，导致高 Peclet 数下失稳；从差分角度，对流项的中心差分近似在对流占优时会产生数值振荡。

所有稳定化方法的核心本质一致：引入人工扩散增强扩散项占比，恢复双线性形式的椭圆性，从而消除数值振荡；不同方法的差异在于人工扩散的强度，迎风格式扩散过量、精度损失大，SUPG 取最优参数实现了稳定与精度的平衡。

7 网格收敛性分析

固定全局 Peclet 数 $Pe_{global} = 120$ （保持物理参数不变），分别取单元数 $nel=10,20,40,80$ ，研究网格加密对数值解的影响规律。

7.1 误差统计

不同网格密度下的单元 Peclet 数、两种方法的最大节点误差统计如表格 3 所示。

表格 3 不同网格密度下的计算误差对比

单元数 nel	单元长度 l_e	单元 Pe	Galerkin 最大误差	SUPG 最大误差
10	0.1000	6.00	7.756798e-01	1.687120e-17
20	0.0500	3.00	5.024802e-01	1.023487e-16
40	0.0250	1.50	2.497871e-01	8.049117e-16
80	0.0125	0.75	8.027302e-02	8.049117e-16

7.2 误差收敛规律

两种方法的最大误差随单元长度的变化规律如图 3 所示，双对数坐标下可直观观察收敛阶次。

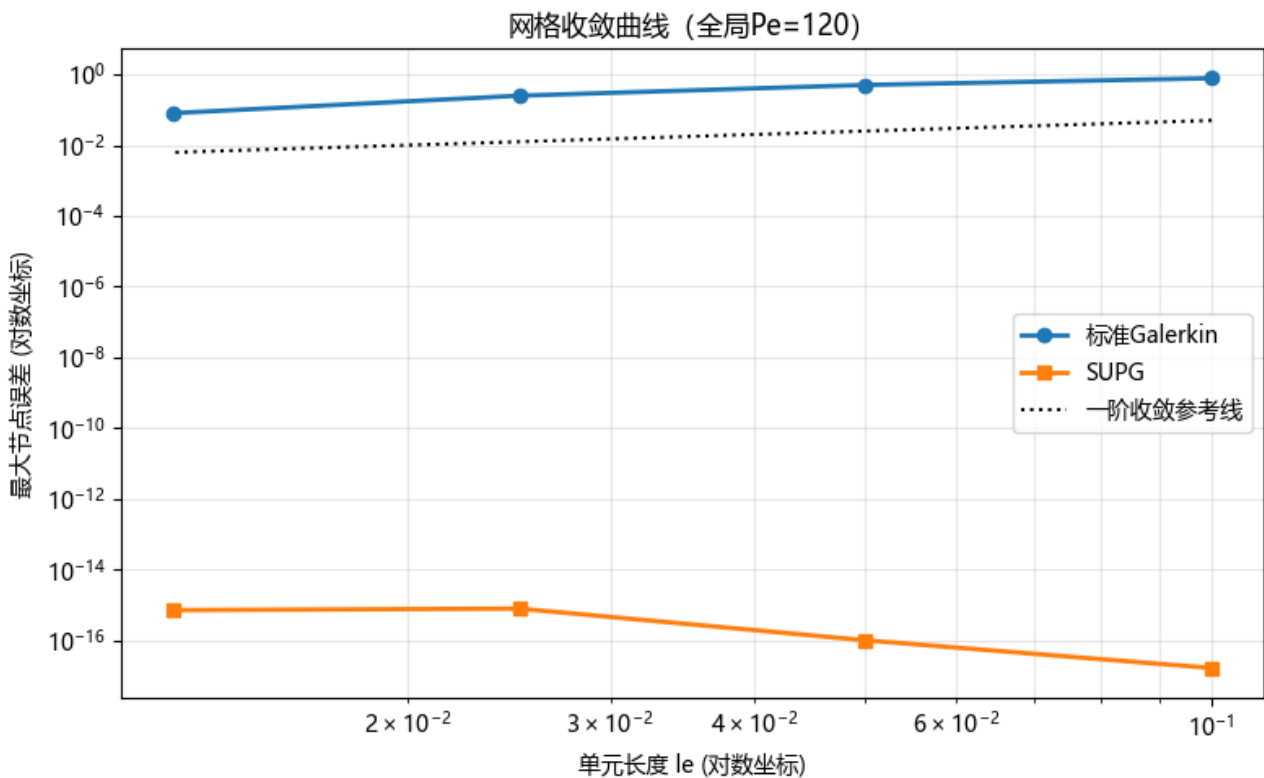


图 3 网格误差收敛双对数曲线

收敛规律分析：

1. 随着网格加密，单元长度减小，单元 Peclet 数逐步降低，Galerkin 方法的振荡逐渐减弱，误差快速下降，最终呈现一阶收敛特性；
2. SUPG 方法在所有网格密度下均保持节点精确性，误差始终处于浮点舍入级别，粗网格下的精度优势尤为显著；
3. 当网格足够致密、单元 $Pe < 1$ 时，Galerkin 方法也可得到稳定结果，但达到相同精度所需的网格量远大于 SUPG 方法。

7.3 Galerkin 解的振荡演变

不同网格密度下 Galerkin 解的对比如图 4 所示，可直观观察到振荡幅度随网格加密逐渐衰减的过程。

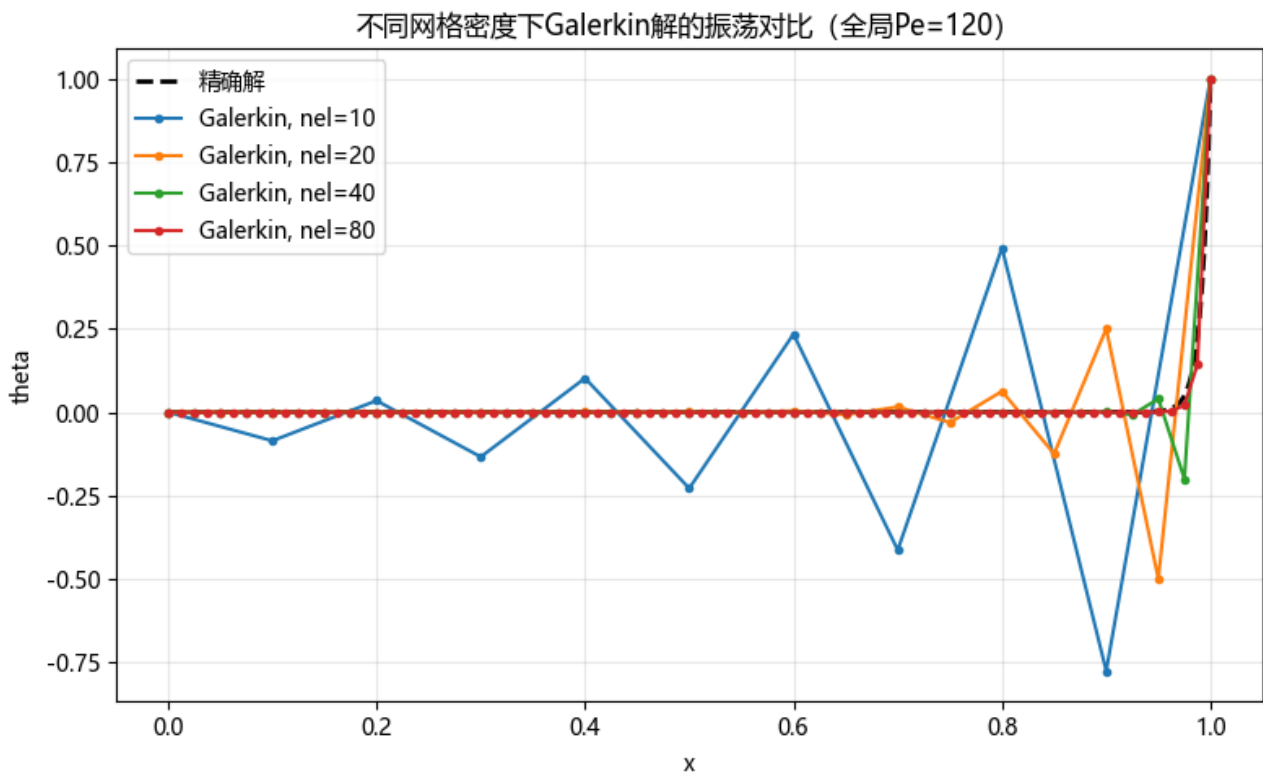


图 4 不同网格密度下 Galerkin 解的振荡对比

8 结论

本文针对一维对流扩散方程，实现了三种有限元离散格式并开展系统对比研究，得到以下结论：

1. 扩散主导工况 ($Pe < 1$) 下，标准 Galerkin 方法可得到稳定、高精度的数值结果；对流占优工况 ($Pe > 1$) 下，标准 Galerkin 方法出现明显的空间数值振荡，解失去物理意义。
2. 迎风格式通过引入人工扩散可完全消除数值振荡，但会带来显著的数值耗散，导致边界层被过度抹平，解的精度下降明显。
3. SUPG 方法通过最优稳定参数自适应调整人工扩散强度，在所有 Peclet 数下均能保持解的稳定性，同时在线性单元下实现了节点精确解，兼顾了稳定性与计算精度，是三种方法中综合性能最优的方案。
4. 网格加密可降低单元 Peclet 数，逐步消除 Galerkin 方法的数值振荡，误差呈一阶收敛；但 SUPG 方法在粗网格下即可获得极高精度，计算效率优势显著。

9 附录

9.1 程序运行结果图

一维对流扩散方程有限元求解程序

课程：3.8 对流扩散方程 程序设计作业

=====

>>> 算例1：单元 $Pe = 0.1$ （扩散主导）

>>> 算例2：单元 $Pe = 3.0$ （对流占优）

=====

三种格式最大节点误差对比表

=====

方法	$Pe=0.1$	$Pe=3.0$
标准Galerkin	1.094240e-03	5.024802e-01
迎风格式	2.977253e-02	1.403784e-01
SUPG	2.666553e-16	1.023487e-16

=====

矩阵性质分析（标准Galerkin，单元 $Pe=3.0$ ，未施加边界）

=====

1. 对称性检测：最大不对称差值 = 1.00e+00

结论：矩阵非对称，由对流项的非对称性导致

2. 正定性检测：最小实特征值 = -0.000000

结论：矩阵半正定（存在零特征值，对应常数齐次解）

3. 对流项对矩阵性质与数值振荡的影响：

（1）对称性：扩散项刚度矩阵对称，对流项刚度矩阵非对称，叠加后整体非对称；

（2）椭圆性：扩散项保证双线性形式的椭圆性，对流占优（ $Pe>1$ ）时椭圆性丧失，弱形式不适定，数值解出现空间振荡；

（3）稳定化本质：引入人工扩散增强扩散项占比，恢复椭圆性，消除振荡；

迎风格式扩散过量导致精度损失，SUPG取最优 α 兼顾稳定与精度。

=====

=====				
附加题：网格收敛性分析（全局 $Pe=120$ ，固定物理参数）				
=====				
单元数 nel	单元长度 le	单元 Pe	Galerkin最大误差	SUPG最大误差

10	0.1000	6.00	7.756798e-01	1.687120e-17
20	0.0500	3.00	5.024802e-01	1.023487e-16
40	0.0250	1.50	2.497871e-01	8.049117e-16
80	0.0125	0.75	8.027302e-02	7.285839e-16
=====				

网格收敛分析结论：

1. 随着网格加密（单元数增加、单元长度减小），单元 $Peclet$ 数逐步降低；
2. Galerkin方法：单元 $Pe>1$ 时存在明显空间振荡，误差较大；当单元 $Pe<1$ 后振荡消失，误差快速下降；
3. SUPG方法：在所有网格密度下均无振荡，节点值为精确解，误差仅为浮点舍入级别；
4. 网格足够密时，Galerkin误差随单元长度一阶收敛，SUPG始终保持最高精度。

=====

9.2 源程序代码

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  from matplotlib import rcParams
4
5  # =====
6  # 全局设置：彻底解决中文+负号显示问题
7  # =====
8  # 优先使用系统自带中文字体，Windows下微软雅黑/黑体均可正常显示
9  rcParams['font.sans-serif'] = ['Microsoft YaHei', 'SimHei', 'Arial Unicode MS', 'DejaVu Sans']
10 # 强制使用ASCII减号，彻底避免Unicode负号字体缺失警告
11 rcParams['axes.unicode_minus'] = False
12 rcParams['text.usetex'] = False
13
14
15 # =====
16 # 1. SUPG 最优稳定参数计算
17 # =====
18 def alpha_supg(Pe): 2用法
19     """
20     计算SUPG/Petrov-Galerkin最优稳定系数
21     公式:  $\alpha = \coth(Pe) - 1/Pe$ 
22      $Pe \rightarrow 0$ 时自动返回0，避免除以零
23     """
24     if abs(Pe) < 1e-12:
25         return 0.0
26     return 1.0 / np.tanh(Pe) - 1.0 / Pe
27
28

```

```

29 # =====
30 # 2. 单元刚度矩阵
31 # =====
32 def element_matrix(kappa, v, le, alpha): 1 个用法
33     """
34     生成两节点线性单元的对流扩散单元矩阵
35     采用作业指定的等效人工扩散格式:
36      $\kappa_{\text{bar}} = \kappa + \alpha * v * le / 2$ 
37      $K_e = \text{扩散项} + \text{对流项}$ 
38     """
39     # 等效扩散系数 (人工扩散叠加物理扩散)
40     kappa_bar = kappa + alpha * v * le / 2.0
41
42     # 扩散项刚度矩阵
43     K_diff = (kappa_bar / le) * np.array([[1, -1],
44                                           [-1, 1]])
45
46     # 对流项刚度矩阵
47     K_conv = (v / 2.0) * np.array([[-1, 1],
48                                   [-1, 1]])
49
50     return K_diff + K_conv
51
52
53 # =====
54 # 3. 组装未加边界的总体刚度矩阵
55 # =====
56 def assemble_global_matrix(nel, L, v, kappa, alpha): 2 用法
57     """
58     组装完整的总体刚度矩阵 (未施加边界条件, 用于矩阵性质分析)
59     """
60     nn = nel + 1
61     le = L / nel
62     K = np.zeros((nn, nn))
63     Ke = element_matrix(kappa, v, le, alpha)
64
65     for e in range(nel):
66         nodes = [e, e + 1]
67         for i in range(2):
68             for j in range(2):
69                 K[nodes[i], nodes[j]] += Ke[i, j]
70     return K
71

```

```

72
73 # =====
74 # 4. 精确解 (数值稳定写法)
75 # =====
76 def exact_solution(x, v, kappa, L): 3 用法
77     """
78     一维稳态对流扩散方程解析解
79     大Peclet数下采用渐近形式避免指数溢出
80     """
81     Pe_global = v * L / kappa
82     if Pe_global > 700:
83         # 强对流渐近形式: 边界层集中在右端
84         return np.exp(v * (x - L) / kappa)
85     else:
86         return (np.exp(v * x / kappa) - 1) / (np.exp(Pe_global) - 1)
87
88
89 # =====
90 # 5. 施加Dirichlet边界条件
91 # =====
92 def apply_bc(K, F, left_val, right_val): 1 个用法
93     """
94     正确施加狄利克雷边界条件: 仅修改边界对应行, 保留列耦合关系
95     """
96     K = K.copy()
97     F = F.copy()
98
99     # 左边界 x=0
100     K[0, :] = 0
101     K[0, 0] = 1
102     F[0] = left_val
103
104     # 右边界 x=L
105     K[-1, :] = 0
106     K[-1, -1] = 1
107     F[-1] = right_val
108
109     return K, F

```



```

110
111
112 # =====
113 # 6. 主求解函数
114 # =====
115 def solve_advection_diffusion(nel, L, v, kappa, alpha): 4用法
116     """
117     完整求解一维对流扩散方程
118     输入: 单元数、域长、对流速度、扩散系数、稳定参数
119     输出: 节点坐标、数值解、精确解、原始总体刚度矩阵
120     """
121     nn = nel + 1
122     le = L / nel
123     x = np.linspace(start=0, L, nn)
124
125     # 组装原始总体矩阵
126     K_raw = assemble_global_matrix(nel, L, v, kappa, alpha)
127     F = np.zeros(nn)
128
129     # 施加边界条件
130     K, F = apply_bc(K_raw, F, left_val=0.0, right_val=1.0)
131
132     # 求解线性方程组
133     theta = np.linalg.solve(K, F)
134
135     # 计算节点处精确解
136     theta_exact = exact_solution(x, v, kappa, L)
137
138     return x, theta, theta_exact, K_raw
139
140
141 # =====
142 # 7. 单算例计算+绘图
143 # =====
144 def run_case(Pe_target, nel=20, L=1.0, v=1.0, save_fig=False): 2用法
145     """
146     运行指定单元Peclet数的算例, 对比三种方法并绘图
147     返回各方法的最大误差字典
148     """
149     le = L / nel
150     kappa = v * le / (2 * Pe_target) # 由单元Pe反算扩散系数

```

```

151
152     # 三种方法的alpha参数
153     methods = {
154         "标准Galerkin": 0.0,
155         "迎风格式": 1.0,
156         "SUPG": alpha_supg(Pe_target)
157     }
158
159     plt.figure(figsize=(8, 5))
160
161     # 绘制高分辨率精确解曲线
162     x_dense = np.linspace(start=0, L, num=500)
163     theta_exact_dense = exact_solution(x_dense, v, kappa, L)
164     plt.plot(*args: x_dense, theta_exact_dense, "k--", linewidth=2, label="精确解")
165
166     error_dict = {}
167     for name, alpha in methods.items():
168         x, theta, theta_exact, _ = solve_advection_diffusion(nel, L, v, kappa, alpha)
169         max_err = np.max(np.abs(theta - theta_exact))
170         error_dict[name] = max_err
171         plt.plot(*args: x, theta, marker="o", markersize=4,
172                 label=f"{name}, 最大误差={max_err:.2e}")
173
174     plt.title(f"一维对流扩散数值解对比 (单元Pe={Pe_target}, 单元数nel={nel}) ")
175     plt.xlabel("x")
176     plt.ylabel("theta")
177     plt.legend()
178     plt.grid(visible=True, alpha=0.3)
179     plt.tight_layout()
180
181     if save_fig:
182         plt.savefig(*args: f"Pe_{Pe_target}.png", dpi=300, bbox_inches="tight")
183     plt.show()
184
185     return error_dict
186
187
188     # =====
189     # 8. 矩阵性质分析
190     # =====
191     def matrix_analysis(nel=20, L=1.0, v=1.0, Pe_target=3.0): 1个用法
192         """
193         分析Pe=3.0下标准Galerkin总体矩阵的性质 (未施加边界条件)
194         """
195         le = L / nel
196         kappa = v * le / (2 * Pe_target)
197         K = assemble_global_matrix(nel, L, v, kappa, alpha=0.0)
198

```

```

199     print("\n" + "=" * 60)
200     print(f"矩阵性质分析（标准Galerkin, 单元Pe={Pe_target}, 未施加边界）")
201     print("=" * 60)
202
203     # 1. 对称性检测
204     sym_error = np.max(np.abs(K - K.T))
205     print(f"1. 对称性检测：最大不对称差值 = {sym_error:.2e}")
206     if sym_error < 1e-10:
207         print("    结论：矩阵对称")
208     else:
209         print("    结论：矩阵非对称，由对流项的非对称性导致")
210
211     # 2. 特征值与正定性检测
212     eigvals = np.linalg.eigvals(K)
213     min_real_eig = np.min(np.real(eigvals))
214     print(f"2. 正定性检测：最小实特征值 = {min_real_eig:.6f}")
215     if min_real_eig > 1e-10:
216         print("    结论：矩阵正定")
217     elif min_real_eig > -1e-10:
218         print("    结论：矩阵半正定（存在零特征值，对应常数齐次解）")
219     else:
220         print("    结论：矩阵存在负实特征值，双线性形式丧失椭圆性")
221
222     # 3. 对流项的影响与振荡原因
223     print("\n3. 对流项对矩阵性质与数值振荡的影响：")
224     print("    (1) 对称性：扩散项刚度矩阵对称，对流项刚度矩阵非对称，叠加后整体非对称；")
225     print("    (2) 椭圆性：扩散项保证双线性形式的椭圆性，对流占优（Pe>1）时椭圆性丧失，")
226     print("        弱形式不适定，数值解出现空间振荡；")
227     print("    (3) 稳定化本质：引入人工扩散增强扩散项占比，恢复椭圆性，消除振荡；")
228     print("        迎风格式扩散过量导致精度损失，SUPG取最优alpha兼顾稳定与精度。")
229     print("=" * 60)
230
231     return K
232
233

```

```

234 # =====
235 # 9. 附加题: 网格收敛性分析
236 # =====
237 def grid_convergence_study(L=1.0, v=1.0, Pe_global=120, save_fig=False): 1 个用法
238     """
239     附加题: 改变单元数研究网格加密对解的影响
240     固定全局Peclet数, 单元数取 [10, 20, 40, 80]
241     分析Galerkin振荡变化、SUPG误差收敛规律, 绘制误差收敛曲线
242     """
243     nel_list = [10, 20, 40, 80]
244     kappa = v * L / Pe_global # 固定物理参数, 全局Pe恒定
245
246     galerkin_errors = []
247     supg_errors = []
248     le_list = []
249     pe_elem_list = []
250
251     print("\n" + "=" * 70)
252     print(f"附加题: 网格收敛性分析 (全局Pe={Pe_global}, 固定物理参数)")
253     print("=" * 70)
254     print(f"{'单元数nel':<10} {'单元长度le':<12} {'单元Pe':<10} {'Galerkin最大误差':<18} {'SUPG最大误差':<18}")
255     print("-" * 70)
256
257     # 遍历不同网格密度
258     for nel in nel_list:
259         le = L / nel
260         pe_elem = v * le / (2 * kappa)
261         le_list.append(le)
262         pe_elem_list.append(pe_elem)
263
264
265     # Galerkin误差
266     _, theta_gal, theta_exact, _ = solve_advection_diffusion(nel, L, v, kappa, alpha=0.0)
267     err_gal = np.max(np.abs(theta_gal - theta_exact))
268     galerkin_errors.append(err_gal)
269
270     # SUPG误差
271     alpha_opt = alpha_supg(pe_elem)
272     _, theta_supg, theta_exact, _ = solve_advection_diffusion(nel, L, v, kappa, alpha=alpha_opt)
273     err_supg = np.max(np.abs(theta_supg - theta_exact))
274     supg_errors.append(err_supg)
275
276     print(f"{'nel':<10} {'le':<12.4f} {'pe_elem':<10.2f} {'err_gal':<18.6e} {'err_supg':<18.6e}")
277     print("=" * 70)
278
279     # 绘制双对数误差收敛曲线
280     plt.figure(figsize=(8, 5))
281     plt.loglog(*args: le_list, galerkin_errors, 'o-', linewidth=2, markersize=6, label='标准Galerkin')
282     plt.loglog(*args: le_list, supg_errors, 's-', linewidth=2, markersize=6, label='SUPG')

```

```

283     # 标注一阶收敛参考线
284     ref_line = 0.5 * np.array(le_list)
285     plt.loglog(*args: le_list, ref_line, 'k:', linewidth=1.5, label='一阶收敛参考线')
286
287     plt.xlabel('单元长度 le (对数坐标)')
288     plt.ylabel('最大节点误差 (对数坐标)')
289     plt.title(f'网格收敛曲线 (全局Pe={Pe_global})')
290     plt.legend()
291     plt.grid(visible=True, which='both', alpha=0.3)
292     plt.tight_layout()
293
294     if save_fig:
295         plt.savefig(*args: "网格收敛曲线.png", dpi=300, bbox_inches="tight")
296     plt.show()
297
298
299     # 绘制不同网格下Galerkin解的对比图 (观察振荡变化)
300     plt.figure(figsize=(8, 5))
301     x_dense = np.linspace(start=0, L, num=500)
302     theta_exact_dense = exact_solution(x_dense, v, kappa, L)
303     plt.plot(*args: x_dense, theta_exact_dense, 'k--', linewidth=2, label='精确解')
304
305     for nel in nel_list:
306         x, theta, _, _ = solve_advection_diffusion(nel, L, v, kappa, alpha=0.0)
307         plt.plot(*args: x, theta, marker='o', markersize=3, label=f'Galerkin, nel={nel}')
308
309     plt.xlabel('x')
310     plt.ylabel('theta')
311     plt.title(f'不同网格密度下Galerkin解的振荡对比 (全局Pe={Pe_global})')
312     plt.legend()
313     plt.grid(visible=True, alpha=0.3)
314     plt.tight_layout()
315
316     if save_fig:
317         plt.savefig(*args: "Galerkin网格加密对比.png", dpi=300, bbox_inches="tight")
318     plt.show()
319
320     # 结果分析说明
321     print("\n网格收敛分析结论: ")
322     print("1. 随着网格加密 (单元数增加、单元长度减小), 单元Peclet数逐步降低; ")
323     print("2. Galerkin方法: 单元Pe>1时存在明显空间振荡, 误差较大; 当单元Pe<1后振荡消失, 误差快速下降; ")
324     print("3. SUPG方法: 在所有网格密度下均无振荡, 节点值为精确解, 误差仅为浮点舍入级别; ")
325     print("4. 网格足够密时, Galerkin误差随单元长度一阶收敛, SUPG始终保持最高精度。")
326     print("=" * 70)

```

```
327 # 10. 主程序
328 # =====
329 if __name__ == "__main__":
330     print("一维对流扩散方程有限元求解程序")
331     print("课程: 3.8 对流扩散方程 程序设计作业")
332     print("=" * 60)
333
334     # 任务2: 两种Peclet数算例
335     print("\n>>> 算例1: 单元Pe = 0.1 (扩散主导)")
336     err_pe01 = run_case( Pe_target: 0.1, save_fig=True)
337
338     print("\n>>> 算例2: 单元Pe = 3.0 (对流占优)")
339     err_pe3 = run_case( Pe_target: 3.0, save_fig=True)
340
341     # 输出误差对比表
342     print("\n" + "=" * 60)
343     print("三种格式最大节点误差对比表")
344     print("=" * 60)
345     print(f"{'方法':<12} {'Pe=0.1':<16} {'Pe=3.0':<16}")
346     print("-" * 60)
347     for method in err_pe01.keys():
348         print(f"{'method':<12} {err_pe01[method]:<16.6e} {err_pe3[method]:<16.6e}")
349     print("=" * 60)
350
351     # 任务4: 矩阵性质分析
352     matrix_analysis()
353
354     # 附加题: 网格收敛性分析
355     grid_convergence_study(save_fig=True)
```